

Gaussian orthant probabilities with BootLoops: Mathematical foundations and implementation

Implementation documentation
(Dated: 4 October 2026)

The Gaussian probability solver evaluates rectangular events by three routes: a closed differential system for integrals over the faces of a box, an auxiliary-field integral for exchangeable precision matrices, and a conditional common-factor integral for equal nonnegative correlations. This document derives the reductions, explains how their numerical evaluation uses BootLoops, and identifies the exact symbolic operations performed by SymPy. It also describes the precision checks and the optional compiled arithmetic. The derivations document the current source; the recently added performance changes have not been executed or validated. Existing numerical receipts belong to the earlier implementation.

I. THE PROBABILITY AND ITS RELATION TO INTEGRAL TRANSPORT

Let X have a d -dimensional normal distribution with mean μ and positive-definite covariance matrix Σ . Write $Q = \Sigma^{-1}$ for its precision matrix. After centering, the bounds become $\ell_i = L_i - \mu_i$ and $u_i = U_i - \mu_i$. The quantity computed is

$$p = \mathbb{P}(L \leq X \leq U) = \frac{\sqrt{\det Q}}{(2\pi)^{d/2}} \int_{\ell \leq x \leq u} \exp\left(-\frac{1}{2}x^\top Q x\right) d^d x. \quad (1)$$

The inequalities are componentwise. A positive orthant has $L_i = 0$ and $U_i = +\infty$; arbitrary finite or infinite endpoints give a box event. The public functions accept either a covariance matrix or a precision matrix, and return both p and its natural logarithm.

The general algorithm follows the integration route described in Section 3.2 of the supplied BootLoops paper [1]: choose a family of integrals, reduce parameter derivatives to a finite set of master integrals, and solve their differential equations from a known boundary value. Integration-by-parts reduction and differential equations are established methods for Feynman integrals [2, 3]. The auxiliary mass flow package of Liu and Ma [4], cited at [arXiv:2201.11669](https://arxiv.org/abs/2201.11669), is a particularly clear example of this strategy: an auxiliary mass parameter connects a tractable limiting problem to the desired integral through a rational differential system. In the present Gaussian problem, the continuation parameter changes correlations and the starting point consists of independent coordinates. No auxiliary mass parameter is introduced, and the executable solver does not call the auxiliary mass flow package.

There is also directly relevant statistical literature: Koyama and Takemura evaluate Gaussian orthant probabilities through a holonomic differential system [5]. The face-integral construction below belongs to this same broad class of finite differential reductions. It is a Gaussian adaptation using BootLoops numerical engines, rather than a claim that scattering-amplitude identities transfer to a Gaussian integrand unchanged. In particular, the code does not construct a scattering alphabet, fit a polylogarithmic function space, or recognize constants through integer relations.

II. CLOSING THE GAUSSIAN FAMILY UNDER DIFFERENTIATION

A. A positive-definite path with an independent starting point

Define $D = \text{diag}(Q_{11}, \dots, Q_{dd})$ and $E = Q - D$. The implemented deformation is $Q(t) = D + tE$, with $0 \leq t \leq 1$. Thus $Q(0) = D$ and $Q(1) = Q$. Every intermediate matrix is positive definite: for

nonzero v , the quadratic form $v^\top Q(t)v = (1-t)v^\top Dv + tv^\top Qv$ is positive. This gives an admissible real continuation path with an elementary independent-coordinate boundary condition.

B. Face integrals are the master integrals

A face F specifies which coordinates are free, fixed at a finite lower endpoint, or fixed at a finite upper endpoint. Let A be its free-coordinate set, B its fixed-coordinate set, and b the vector of fixed values. Its unnormalized master integral is

$$J_F(t) = \int_{\ell_A \leq x_A \leq u_A} \exp \left[-\frac{1}{2} \begin{pmatrix} x_A \\ b \end{pmatrix}^\top \begin{pmatrix} Q_{AA}(t) & Q_{AB}(t) \\ Q_{BA}(t) & Q_{BB}(t) \end{pmatrix} \begin{pmatrix} x_A \\ b \end{pmatrix} \right] dx_A. \quad (2)$$

When no coordinates are free, this is simply the Gaussian exponential evaluated at a corner. The full-box integral is $J_\emptyset$. Infinite endpoints do not create fixed faces: their Gaussian boundary contributions vanish.

Differentiating J_F with respect to t produces first and second moments over the same face. For free indices, define $m_i = \int_F x_i f_t dx_A$ and $M_{ij} = \int_F x_i x_j f_t dx_A$, where f_t is the exponential in Eq. (2). These moments can be eliminated without adding new master integrals.

C. Integration by parts, including the boundary terms

Set $R = Q_{AA}(t)^{-1}$ and $c = Q_{AB}(t)b$. For a free coordinate j , let F_j^+ and F_j^- be its upper and lower child faces, omitting an infinite endpoint. Put $\Delta_j = J_{F_j^+} - J_{F_j^-}$, with a missing face contributing zero. Integrating $\partial f_t / \partial x_j$ gives $\Delta_j = -\sum_k Q_{jk}(t)m_k - c_j J_F$.

For the second moment, let H_{ji} be the upper-minus-lower boundary integral of $x_i f_t$ on coordinate j . Integrating $\partial(x_i f_t) / \partial x_j$ gives $H_{ji} = \delta_{ji} J_F - \sum_k Q_{jk}(t)M_{ki} - c_j m_i$. Consequently,

$$m = -R(\Delta + cJ_F), \quad M = R(J_F I - cm^\top - H). \quad (3)$$

If $i = j$, the boundary value in H_{ji} is the endpoint times a child master. If $i \neq j$, it is a first moment of a child face, which is reduced by the same first identity. Each recursive boundary operation fixes another coordinate, so it eventually terminates at corner values. Keeping these boundary terms is essential for finite boxes and orthants.

Substitution into the parameter derivative yields

$$J'_F = -\frac{1}{2} \text{tr}(E_{AA}M) - (E_{AB}b)^\top m - \frac{1}{2} b^\top E_{BB}b J_F. \quad (4)$$

Equations (3) and (4) express every derivative as a linear combination of face masters. Assembling all faces therefore gives $J'(t) = \mathcal{A}(t)J(t)$ with rational coefficients in t . This is the differential system implemented by `BoundarySystem` in `transport.py`. The face family provides a finite closed system; the code does not prove that it is a minimal basis.

D. Dimension and permutation symmetry

For a general fully bounded box, each coordinate has three face states, so the system contains 3^d masters. With one finite endpoint per coordinate it contains 2^d masters. More generally the count is the product of the available face-state counts. The default resource cap is 256 masters, checked before enumerating the faces.

When the diagonal entries are equal, the off-diagonal entries are equal, and all lower bounds and all upper bounds are respectively identical, coordinate permutations identify equivalent faces. A fully bounded face then depends only on the numbers of free, lower-fixed, and upper-fixed coordinates. The master count becomes $\binom{d+2}{2}$. This symmetry reduction is distinct from the one-dimensional structured formulas derived below.

III. WHAT SYMPY DOES

SymPy is the exact algebra layer [6]. Input decimal strings and fractions are converted to rational numbers, and the mean is subtracted from the endpoints before conversion to numerical arithmetic. Ordinary floating-point inputs retain their supplied decimal representation, not unknown digits of an underlying physical parameter.

The symbolic construction forms $Q(t)$, inverts its free-coordinate principal submatrices, and performs the matrix products and traces in Eqs. (3) and (4). It cancels rational expressions and extracts numerator and denominator polynomial coefficients for each nonzero entry of $\mathcal{A}(t)$. The principal-submatrix inverses and face-moment identities are reused during construction. Covariance inversion and the determinant used in the probability normalization are also exact symbolic operations. If numerical matrix validation loses a positive pivot through rounding, the code checks positive definiteness with exact rational arithmetic; diagonal matrices are checked directly by the signs of their entries.

The Gaussian seeds, error functions, eigenvalue estimates used for step control, Taylor stepping, and quadrature are numerical operations. SymPy is not asked to find a symbolic antiderivative of the box integral. After construction, the numerical engine receives sparse polynomial data, so symbolic simplification does not occur inside each Taylor step.

IV. BOUNDARY CONDITIONS AND THE BOOTLOOPS TRANSPORT

A. Exact-form seeds and numerical scaling

Let Φ and ϕ denote the standard normal distribution function and density. At $t = 0$, free coordinates factor, giving the seed

$$s_F = J_F(0) = \prod_{i \in A} \sqrt{\frac{2\pi}{D_{ii}}} [\Phi(u_i \sqrt{D_{ii}}) - \Phi(\ell_i \sqrt{D_{ii}})] \exp\left(-\frac{1}{2} \sum_{j \in B} D_{jj} b_j^2\right). \quad (5)$$

These are known univariate formulas, evaluated at the working precision. The code computes each coordinate's interval mass and endpoint density once and reuses them across faces.

With $S = \text{diag}(s_F)$ and $W = S^{-1}J$, the initial vector is $W(0) = \mathbf{1}$ and the connection becomes $W' = \mathcal{C}(t)W$, where $\mathcal{C} = S^{-1}\mathcal{A}S$. An entry is multiplied by s_G/s_F , not its reciprocal. This scaling reduces the disparity between initial master magnitudes. After transport, Eq. (1) becomes $p = \sqrt{\det Q} s_\emptyset W_\emptyset(1)/(2\pi)^{d/2}$. The determinant is reused across the higher-precision runs.

B. Taylor continuation along the correlation path

The numerical call is `BootLoops' wayfinder.transport.transport_fixed_eps`. Its deformation argument is fixed to zero; here that argument has no dimensional-regularization meaning.

About a point t_0 , write $\mathcal{C}(t_0 + h) = \sum_{k \geq 0} C_k h^k$ and $W(t_0 + h) = \sum_{m \geq 0} w_m h^m$. Equating powers gives

$$w_{m+1} = \frac{1}{m+1} \sum_{k=0}^m C_k w_{m-k}. \quad (6)$$

BootLoops advances by local series, checks the decay of the terms, reduces a step when its numerical tail estimate is too large, and continues to $t = 1$. Its acceptance logic accounts for intervening zero Taylor terms instead of treating one small term as sufficient evidence of convergence. The Gaussian adapter supplies the local connection series directly, avoiding the generic recovery of series coefficients from samples on a complex circle.

The possible finite poles come from $\det Q_{AA}(t) = 0$. Factoring out D_{AA} gives $\det Q_{AA}(t) = \det D_{AA} \det(I + tB_A)$, with $B_A = D_{AA}^{-1/2} E_{AA} D_{AA}^{-1/2}$. Its eigenvalues are real, and each nonzero eigenvalue λ gives a possible pole $t = -1/\lambda$. These locations are estimated with arbitrary-precision symmetric eigenvalue calculations and passed to the engine for step control. Positive definiteness excludes a pole on the chosen real interval. Some listed poles can be apparent or cancel in a particular entry.

For a rational connection entry $n(h)/q(h)$, the reciprocal denominator series starts at $v_0 = 1/q_0$ and obeys $v_k = -q_0^{-1} \sum_{j=1}^k q_j v_{k-j}$, treating coefficients above the polynomial degree as zero. Multiplying by the shifted numerator gives the desired connection series. The implementation in `_series.py` inverts a shared denominator once per expansion point and keeps only nonzero matrix entries. The local BootLoops adapter accepts this sparse interface and retains real arithmetic on real paths. Complex waypoint detours use complex arithmetic.

C. Optional compiled arithmetic

The Python interfaces retain their existing mathematical definitions.

The default, `backend="auto"`, selects `python-flint` when available, and otherwise uses `mpmath`. An explicit `flint` selection requires the optional package. The compiled transport uses BootLoops' `AcbFastTable` and its polynomial-convolution Taylor engine. Gaussian interval kernels also evaluate complementary error functions in compiled real and complex ball arithmetic [7, 8].

Conversions preserve the stored binary mantissa and exponent instead of passing through machine-precision numbers. The interval kernels increase their precision until the local ball resolves the requested working bits. However, the transport engine returns ball midpoints between accepted steps; it does not propagate a rigorous enclosure through the entire calculation. The complete probability therefore remains subject to the numerical precision and refinement checks described in Section VII. Arithmetic contexts are restored after calls. Concurrent use of the shared contexts in separate threads is unsupported; separate processes are the appropriate route for concurrent evaluations.

V. EXCHANGEABLE PRECISION MATRICES: AN AUXILIARY FIELD

A. Factorization of the quadratic coupling

The function `exchangeable_precision_probability` treats $Q = aI + b\mathbf{1}\mathbf{1}^\top$, with $a > 0$, $b \geq 0$, and common finite centered bounds $\ell < u$. Its determinant is $a^{d-1}(a + bd)$. This is a precision-matrix structure: its covariance is $a^{-1}I - b\mathbf{1}\mathbf{1}^\top/[a(a + bd)]$, so positive b produces negative off-diagonal covariances. It should not be confused with the positive-correlation model of the next section.

Put $T_x = \sum_i x_i$. The Gaussian Fourier form of the Hubbard–Stratonovich transformation [9] is $\exp(-bT_x^2/2) = (2\pi b)^{-1/2} \int_{\mathbb{R}} \exp[-y^2/(2b) + iyT_x] dy$. It converts the coupling into a single auxiliary variable and makes the coordinate integrals factor. Define $I_a(s) = \int_{\ell}^u \exp(-ax^2/2 - sx) dx$. Completing the square expresses it through complementary error functions:

$$I_a(s) = \sqrt{\frac{\pi}{2a}} \exp\left(\frac{s^2}{2a}\right) \left[\operatorname{erfc}\left(\frac{a\ell + s}{\sqrt{2a}}\right) - \operatorname{erfc}\left(\frac{au + s}{\sqrt{2a}}\right) \right]. \quad (7)$$

Here s can be complex. Finite bounds make $I_a(s)$ entire, which permits shifting the auxiliary contour to $y + i\lambda$ for real λ . The Gaussian factor controls the ends of the shifted contour.

B. Saddle centering and the remaining integral

The shifted representation is

$$p = \frac{\sqrt{a^{d-1}(a + bd)}}{(2\pi)^{d/2}} \exp\left(\frac{\lambda^2}{2b}\right) I_a(\lambda)^d K_{\lambda}, \quad (8)$$

$$K_{\lambda} = \frac{1}{\sqrt{2\pi b}} \int_{\mathbb{R}} e^{-y^2/(2b)} e^{-i\lambda y/b} \left[\frac{I_a(\lambda - iy)}{I_a(\lambda)} \right]^d dy.$$

Under the tilted one-coordinate density $f_{\lambda}(x) = \exp(-ax^2/2 - \lambda x)/I_a(\lambda)$ on $[\ell, u]$, write $\mu_{\lambda} = \mathbb{E}_{\lambda}[X]$ and $v_{\lambda} = \operatorname{Var}_{\lambda}(X)$. The code chooses $\lambda/b = d\mu_{\lambda}$, making the linear phase at $y = 0$ vanish. Since $d\mu_{\lambda}/d\lambda = -v_{\lambda}$, the derivative of the saddle residual is $1/b + dv_{\lambda} > 0$. The root is bracketed between $bd\ell$ and bdu and found by safeguarded Newton iteration.

The natural width is $w = \sqrt{b/(1 + bdv_{\lambda})}$. Substituting $y = wz$ centers and scales the integrand. Its conjugate symmetry allows integration of its real part over $z \geq 0$, followed by doubling. The potentially very small prefactor is accumulated logarithmically. `BootLoops'` `wayfinder.quad.quad_refine` evaluates the remaining integral using progressively refined quadrature [10].

C. A relative bound on the discarded Fourier tail

For independent draws from f_{λ} , let $T = \sum_i X_i$ and $r_{\lambda} = d\mu_{\lambda} - \lambda/b$. Interchanging the expectation and Gaussian Fourier integral in Eq. (8) gives $K_{\lambda} = \mathbb{E}_{\lambda} \exp[-b(T - \lambda/b)^2/2] > 0$. Jensen's inequality gives $K_{\lambda} \geq \exp[-b(dv_{\lambda} + r_{\lambda}^2)/2]$. The bounded-interval variance inequality $v_{\lambda} \leq (u - \ell)^2/4$ supplies a computable lower bound.

The absolute value of the characteristic function in Eq. (8) is at most one. Cutting the auxiliary variable off at $|y| = Y$ therefore gives the relative discarded-tail bound

$$\frac{|K_{\lambda} - K_{\lambda}^{(|y| \leq Y)}|}{K_{\lambda}} \leq \operatorname{erfc}\left(\frac{Y}{\sqrt{2b}}\right) \exp\left[\frac{b}{2} \left(\frac{d(u - \ell)^2}{4} + r_{\lambda}^2\right)\right]. \quad (9)$$

The implementation retains the residual term and increases the cutoff until this bound meets its target. This controls the omitted tail. Errors in quadrature, arithmetic, and the prefactor are assessed separately. For $b = 0$, or for one coordinate, the code uses the direct univariate Gaussian formulas instead of introducing an auxiliary field.

VI. EQUAL NONNEGATIVE CORRELATIONS: A COMMON GAUSSIAN FACTOR

The function `equicorrelated_probability` treats the representation $X_i = \mu + \sigma(\sqrt{\rho}Z + \sqrt{1-\rho}Z_i)$, where all the standard normal variables are independent and $0 \leq \rho < 1$. Standardize the common endpoints by subtracting μ and dividing by σ , and call them ℓ, u . Conditional independence gives

$$p = \int_{\mathbb{R}} \phi(z)g(z)^d dz, \quad g(z) = \Phi\left(\frac{u - \sqrt{\rho}z}{\sqrt{1-\rho}}\right) - \Phi\left(\frac{\ell - \sqrt{\rho}z}{\sqrt{1-\rho}}\right). \quad (10)$$

This is a positive one-dimensional integral. It supports finite or infinite equal bounds and a common nonzero mean and standard deviation.

To resolve rare, displaced peaks, define $h(z) = \log \phi(z) + d \log g(z)$. Let m_z, v_z be the mean and variance of a standard normal restricted to the conditional interval in Eq. (10), and put $r = \sqrt{\rho/(1-\rho)}$. Direct differentiation gives $h'(z) = -z + dr m_z$ and $h''(z) = -1 - dr^2(1 - v_z)$. The interval-truncated variance satisfies $0 \leq v_z \leq 1$, so the log integrand is strictly concave and has a unique mode. A safeguarded Newton iteration finds the mode z_0 . With $w = 1/\sqrt{-h''(z_0)}$, the code integrates $\exp[h(z_0 + ws) - h(z_0)]$ and restores the factor $w \exp[h(z_0)]$ afterward.

The bound $h'' \geq -K$, where $K = 1 + d\rho/(1-\rho)$, also removes the need for a preliminary integration to estimate the magnitude. For any chosen center, Taylor's integral remainder bounds $h(z_0 + ws) - h(z_0)$ below by $h'(z_0)ws - Kw^2s^2/2$. Integrating this lower Gaussian bound shows that the scaled integral is at least $\sqrt{2\pi/K} \exp[h'(z_0)^2/(2K)]/w$, hence at least $\sqrt{2\pi/K}/w$. The code passes half this last quantity as a conservative tolerance scale to BootLoops quadrature. This argument remains valid when the numerically found mode has a small nonzero residual.

A useful independent truth is the positive orthant at $\rho = 1/2$. Then $g(z) = \Phi(z)$ and the substitution $v = \Phi(z)$ gives $p = \int_0^1 v^d dv = 1/(d+1)$. For $d > 1$ the structured solver evaluates the integral; this identity is used for validation, including large dimensions. At $\rho = 0$, or for a single coordinate, it returns through the direct Gaussian formula.

VII. PRECISION, LOGARITHMS, AND NUMERICAL EVIDENCE

A. Independent formulas and narrow intervals

For diagonal Q , the probability is $\prod_i [\Phi(u_i\sqrt{Q_{ii}}) - \Phi(\ell_i\sqrt{Q_{ii}})]$. The direct path avoids master allocation, including its exponential resource cap, and still requires agreement at different working precisions. Full Gaussian space returns exactly one; a zero-width event returns zero.

Gaussian tails are evaluated with complementary error functions and reflection, avoiding differences of numbers close to one. A very narrow finite interval uses a local series instead of subtracting nearly equal endpoint functions. To see this, write $x = m + vz$ on an interval of midpoint m and half-width v . The exponent becomes a constant plus $\alpha z + \beta z^2$, where $\alpha = -v(am + s)$ and $\beta = -av^2/2$. If $\exp(\alpha z + \beta z^2) = \sum_n c_n z^n$, then $c_0 = 1$, $c_1 = \alpha$, and $(n+1)c_{n+1} = \alpha c_n + 2\beta c_{n-1}$. Integrating over $-1 \leq z \leq 1$ uses only the even coefficients, giving $2 \sum_{k \geq 0} c_{2k}/(2k+1)$. The same expansion provides the moments needed in the saddle searches. Exact endpoint differences determine additional working precision before numerical conversion.

B. Acceptance at increasing precision

Each numerical route reruns the evaluation with additional working digits; the quadrature routes also tighten their integration tolerances. For requested precision k , successive probabilities

must have relative difference below 10^{-k} . For probabilities strictly between zero and one, the code also compares their logarithms relative to the magnitude of the logarithm. The general transport additionally checks the residual imaginary component. Failure to pass the checks raises a convergence error.

When p is near one, an apparently stable rounded value can hide a meaningful small negative $\log p$. The complement of a box contains every marginal complement, giving a lower bound on $1 - p$. Marginal outside tails therefore determine extra working digits needed to resolve the logarithm. The logarithm is computed before the final rounding of p , so it can remain nonzero even if the displayed probability rounds to one.

BootLoops quadrature refines its rule depth until successive estimates agree within the requested tolerance. Its default $\tanh$ - $\sinh$ rule is based on double-exponential quadrature [10]. Local Taylor tail estimates, quadrature refinement, and higher-precision agreement provide numerical evidence. The analytic bound in Eq. (9) covers only the discarded auxiliary tail. These ingredients do not form a rigorous interval enclosure of the complete probability.

C. Comparison with minimax tilting and the saved receipts

Botev’s minimax tilting method [11] constructs a tilted sequential importance-sampling distribution by solving a saddle optimization. It is a statistical comparator to the deterministic reductions here. The one-variable saddle in Eq. (8) is a contour-centering condition, not Botev’s minimax optimization. The project’s independent validation implementation uses twelve scrambled Sobol sequences of 4096 points each, with recorded seeds; those estimator uncertainties are statistical and do not establish all the candidate’s displayed digits.

The saved validation covers the exchangeable-precision Example I in Section 5.1 at dimensions 2, 3, 5, 10, 20, and 50; the banded-precision Example II at dimensions 2 and 3; and the exact equal-correlation family in Section 5.3 at dimensions 10, 100, 1000, and 10000. Example I is also checked against positive convolution of the original integrand and, at the smaller dimensions, boundary transport. Example II has an independent positive product-quadrature check.

In those reports, the positive-convolution relative difference must be below 10^{-10} , the Example I boundary comparison below $10^{-(k-2)}$, and the minimax comparison below the larger of six estimated relative standard errors and 10^{-10} . Example II product quadrature requires agreement below 10^{-12} both with the candidate and between its two rule orders. The exact family requires relative difference below 10^{-k} . A separate comparison with the printed paper allows five printed standard errors plus one last-digit unit and flags discrepancies; it is not the independent correctness gate.

For Example I at dimension 10, the historical candidate is about $8.56248967736 \times 10^{-15}$ whereas the paper prints 8.556×10^{-15} . The receipt preserves this discrepancy alongside the independent comparisons. These are historical results: the current compiled kernels, sparse transport, and other performance changes have not been run. This documentation does not turn the earlier passing checks into validation of the changed source, and no speedup is claimed.

VIII. READING THE IMPLEMENTATION AND ITS PRACTICAL LIMITS

Table I maps the mathematics to the principal source modules. General correlated boxes use the boundary route; the two structured reductions have explicit public interfaces. Recognizing a structured matrix in the general interface can merge equivalent faces, but does not automatically select the auxiliary-field or common-factor function.

TABLE I. Mathematical responsibilities of the source modules. All paths are within the `gaussian_orthant` package unless stated otherwise.

Module	Responsibility
<code>transport.py</code>	Exact face reduction, rational connection, independent seeds, normalization, and the call to BootLoops transport.
<code>structured.py</code>	Auxiliary contour shift, saddle and mode equations, analytic tolerance scales, and calls to BootLoops quadrature.
<code>_series.py</code>	Sparse rational Taylor coefficients with shared denominator inversion.
<code>_kernels.py</code>	Optional compiled Gaussian interval kernels and precision-controlled conversion of ball midpoints.
<code>_independent.py</code>	Products of univariate Gaussian probabilities and their precision checks.
<code>common.py</code>	Exact inputs, interval and tail precision guards, stable local series, logarithm checks, and result objects.
<code>_bootloops.py</code>	Loading the local BootLoops transport, quadrature, and rational-function engines.
<code>validate.py</code>	Independent comparisons, published-table flags, and numerical receipts recording the source and arithmetic backend.

The general cost is controlled first by the number of faces, then by symbolic construction and numerical transport. For s nonzero connection entries and Taylor order M , a direct convolution step entails roughly sM^2 scalar operations. Sparse storage, shared denominator inverses, and real arithmetic reduce unnecessary work; compiled polynomial products accelerate the arithmetic. They do not remove the general exponential growth of the face family. The structured routes avoid that growth, but still depend on quadrature and the evaluation of real or complex Gaussian interval functions. Their applicability is restricted to the matrix families and bounds stated above. No general large-dimensional performance advantage over minimax tilting has been established.

The practical outcome is a finite, explicitly derived Gaussian differential system when the general matrix is small enough, and one-dimensional integrals for the stated structured families. SymPy establishes the rational algebra; BootLoops carries out continuation or refined quadrature; precision checks determine whether a numerical result is accepted. The mathematical identities explain the algorithm, while fresh numerical validation is still required for the recently changed implementation.

-
- [1] Matthew Schwartz, *BootLoops: an LLM-driven toolkit for exact quantitative science*, supplied manuscript dated 1 October 2026, especially Section 3.2. [Manuscript](#); [project documentation](#); [source repository](#).
 - [2] Chetyrkin and Tkachov, *Integration by parts: The algorithm to calculate β -functions in 4 loops*, Nuclear Physics B **192**, 159–204 (1981). [Publisher reference](#).
 - [3] Kotikov, *Differential equations method. New technique for massive Feynman diagram calculation*, Physics Letters B **254**, 158–164 (1991). [Publisher reference](#).
 - [4] Xiao Liu and Yan-Qing Ma, *AMFlow: a Mathematica package for Feynman integrals computation via Auxiliary Mass Flow*, Computer Physics Communications **283**, 108565 (2023). [arXiv:2201.11669](#); [publisher reference](#).
 - [5] Tamio Koyama and Akimichi Takemura, *Calculation of orthant probabilities by the holonomic gradient method*, manuscript (2012; revised 2013). [arXiv:1211.6822](#).
 - [6] Aaron Meurer and collaborators, *SymPy: symbolic computing in Python*, PeerJ Computer Science **3**, e103 (2017). [Publisher reference](#); [authors' source repository](#).
 - [7] Fredrik Johansson, *Arb: Efficient Arbitrary-Precision Midpoint-Radius Interval Arithmetic*, manuscript (2016). [arXiv:1611.02831](#).
 - [8] The python-flint developers, *python-flint documentation*, especially real and complex ball arithmetic and power-series interfaces. [Software documentation](#), consulted 4 October 2026.
 - [9] John Hubbard, *Calculation of Partition Functions*, Physical Review Letters **3**, 77 (1959). [Publisher reference](#).
 - [10] Hidetosi Takahasi and Masatake Mori, *Double Exponential Formulas for Numerical Integration*, Publications of the Research Institute for Mathematical Sciences **9**, 721–741 (1974). [Publisher reference](#).
 - [11] Botev, *The Normal Law Under Linear Restrictions: Simulation and Estimation via Minimax Tilt-ing*, Journal of the Royal Statistical Society Series B: Statistical Methodology **79**, 125–148 (2017). [arXiv:1603.04166](#); [publisher reference](#).